TypeScript for Beginners

Safer JavaScript with types

A concise, original learning guide with practical examples and a mini project.

Designed for beginners • 3 learning pages + cover

1. Why TypeScript?

TypeScript adds a static type system to JavaScript. It helps detect many mistakes during development and improves editor autocomplete and refactoring.

Basic types

```
let username: string = "Riya";\nlet count: number = 5;\nlet isAvailable: boolean = true;\nlet labels: s
tring[] = ["Python", "React"];
```

TypeScript can infer types from initial values, so explicit annotations are most useful when they clarify function contracts or data structures.

Object types

```
type Book = {\n  id: number;\n  title: string;\n  author: string;\n  available: boolean;\n};\n\nconst b
ook: Book = {\n  id: 1, title: "React",\n  author: "A. Shah", available: true\n};
```

A type describes the shape of data. Optional properties use ? and readonly can communicate that a property should not be reassigned.

2. Functions and Union Types

Typed functions

```
function formatTitle(title: string): string {\n  return title.trim().toUpperCase();\n}\n\nfunction find
Book(id: number, books: Book[]): Book | undefined {\n  return books.find(book => book.id === id);\n}
```

Parameter types define expected inputs, and return types define outputs. A function may return undefined when a search finds no matching item.

Union types

```
type Status = "available" | "borrowed" | "reserved";\n\nfunction getStatusLabel(status: Status) {\n  re
turn status.toUpperCase();\n}
```

A union restricts a value to one of several allowed types. Literal unions are useful for statuses, payment modes, and UI variants.

Narrowing

```
function printValue(value: string | number) {\n  if (typeof value === "string") {\n    console.log(valu
e.toUpperCase());\n  } else {\n    console.log(value.toFixed(2));\n  }\n}
```

Type narrowing lets TypeScript understand which type is valid inside a branch.

3. TypeScript in React Projects

Typing component props

```
type BookCardProps = {\n  book: Book;\n  onRead: (id: number) => void;\n};\n\nfunction BookCard({ book,
 onRead }: BookCardProps) {\n  return (\n    <button onClick={() => onRead(book.id)}>\n      Read {book
.title}\n    </button>\n  );\n}
```

Typed props make component contracts clear. Prefer specific types over any, which disables useful type checking.

Working safely

Use unknown when a value's type is not yet known, then validate or narrow it. API data should be validated at runtime when correctness or security matters; TypeScript types disappear after compilation.

Mini project

Create a Book type, a typed array of books, a search function, and a React card component. Add a Status union so invalid availability values are caught during development.

TypeScript improves developer confidence, but it does not replace runtime validation or automated tests.